vesuvius-challenge-surface-detection

1st Place Solution for the Vesuvius Challenge - Surface Detection Competition

Rank	1
Team	Vesuvius Team
Author	PaulG
Topic ID	679238
Version	Original

Thanks to the organizers for this fun and important competition, and congratulations to all the winners!

Context

- **https://www.kaggle.com/competitions/vesuvius-challenge-surface-detection/overview**
- **https://www.kaggle.com/competitions/vesuvius-challenge-surface-detection/data**

Overview of the approach

Our solution consisted of an nnU-Net ensemble plus post-processing.

Details of the submission

nnU-Net

First, we would like to appreciate @jirkaborovec 's work! His notebook served as the starting point of our training pipeline.

Single Model Strategy

- We used all available data for training and built our baseline nnU-Net model (Model 1) with the following settings:

```
patch size: 128
batch size: 2
epochs: 4000
```

- We then fine-tuned Model 1 with larger patch sizes of 192 and 256, training for 250 epochs each.
- Due to the extension of the competition, we trained additional models from scratch (for 4000 epochs) with patch sizes of 160, 192, and 224 to increase model diversity. Among these, the 192-patch model was included in our final ensemble.
- As for the single model performance, , and two of our best single models are (due to submission quota, we only did basic post processing on single model submission for A/B tests, so full post processing may produce better results):

Setting	Public LB	Private LB
fine-tuned 192-patch model at 250 epochs	0.577	0.614
from-scratch 192-patch model at 4000 epochs	0.587	0.613

Ensemble Strategy

We prepared two sets of 4-model ensembles as our final submissions:

- **Set 1:** Baseline 128-patch Model 1 (weight: 0.12) + fine-tuned 192-patch model (0.28) + fine-tuned 256-patch model at 100 epochs (0.18) + fine-tuned 256-patch model at 250 epochs (0.42).
- **Set 2:** Baseline 128-patch Model 1 (weight: 0.12) + fine-tuned 192-patch model (0.28) + fine-tuned 256-patch model at 250 epochs (0.18) + from-scratch 192-patch model at 4000 epochs (0.42).

For each ensemble, we fused the post-softmax probabilities using the assigned model weights and applied different thresholds and post processing methods (will be discussed in the following section).

Ensemble	Public LB	Private LB	Threshold
Set 1	0.613	0.620	0.20
Set 2	0.606	0.627	0.26

Post-processing

As discussed in many of the forum posts, minimizing holes and cavities in the predicted masks was essential for getting a good score. To that end, we applied five post-processing steps:

0. We removed mask components that have less than 20K voxels.

Then for each sheet (where a sheet is a connected-component in a predicted mask), we did the following:

1. Applied scipy.ndimage.binary_closing(), with a spherical footprint of radius 3. This closes holes and cavities of radius 3 or less.

2. Patching: To repair larger holes, we represented each sheet as a height map, and filled any gaps in the height map by linear interpolation across the gap. We did this separately along the x and y directions, and averaged the two results, weighted by distance to the nearest gap-edge. The projection axis for creating the height map was chosen to be the one that gave the largest projected sheet area. This method is capable of significant repairs:

3. We created a simple function to plug any small (1-voxel) remaining holes. For each voxel in a given sheet, we look at it's 2 x 2 x 2 neighborhood and add whatever voxels are needed to make that neighborhood 6-connected watertight. For example:

(This method was inspired by the scikit-image euler_number() function, which uses such neighborhoods to compute the Euler number.) We don't have a proof that our function plugs all 1-voxel holes, but it seems to work well in practice. The algorithm is implemented as a lookup table, with 256 entries for the 256 possible neigborhoods. To create the lookup table, we used a very high tech scotch-tape-and-paper model to visualize all the cases:

(Ordinary dilation will also plug holes, but it tends to add too many voxels, which can degrade the dice score.)

4. Finally, we called scipy.ndimage.binary_fill_holes() on the whole mask, which fills arbitrary-size cavities.

Our best version of this post-processing pipeline also included a check of the number of holes before and after patching, because the patching can actually introduce small holes, for example when the sheet is very curved. In such cases we discard the patch.

The following table shows the cumulative effect of each post-processing step on our best nnU-Net ensemble:

Step	Public Score	Private Score
No post-proc	.572	.596
Remove small components	.586	.614
Plug small holes	.598	.622
Patch large holes	.601	.625
Binary closing	.606	.627
Fill_holes	.606	.627

What didn't work

Unfortunately, we did not find an effective solution to the problem of touching sheets. We just relied on nnu-Net to minimize their occurrence.

What we misled by public LB

- logits fusion performs better on private LB, but we trusted (overfitted) on probability fusion
- larger threshold (Like 0.35-0.4 is the best searched value on our model on training set) performs better on private LB

How Vibe Coding helps

Initially, our ensemble pipeline got resources issues (GPU memory, hard disk space, …) even with 2 models ensemble, ChatGPT helps us redesign the pipeline, and enables 4 models ensemble.

Sources

- **https://www.kaggle.com/code/jirkaborovec/surface-nnunet-training-inference-with-2xt4**
- 0.627 solution notebook

Supplemental Q&A

> **artineon · 2026-02-28T04:33:57.653000**
>
> what is the contribution of threshold tuning?

> **Yiheng Wang · 2026-02-28T05:19:10.383000**

> Impact is much less than post processing, like 1e-3 level. Since we don't have validation set, changing thresholds is just a way we choose to make full use of submission quota

> **Yiheng Wang · 2026-02-28T08:07:18.833000**
>
> add one more comment: it's a place that can be enhanced (updated the writeup). Larger thresholds like 0.35-0.4 perform better on local search & and private LB, but worse on public LB. We overfitted on this part.

> **Giorgio Angelotti · 2026-03-01T09:21:12.487000**
>
> Congrats for the great achievement! ☒
>
> Do you feel that your silver bullet has been the heightmap based post processing?

> **PaulG · 2026-03-01T14:15:01.227000**
>
> Thanks!
>
> The heightmap code targets large holes and, although it's really satisfying to see those big holes disappear, it doesn't lead to much of a score improvement, unfortunately. The reason is that there aren't very many holes like that, and their area, as a fraction of the total area of all the sheets in a volume, is usually small.

> **Arpit1Bansal · 2026-03-04T10:04:06.530000**
>
> Hey, can you share your train/val curves while training? What things let you decide, that this needs to train for 100, 250 even 4000 epochs. which is a large number. And yes congrats to the whole team.

> **PaulG · 2026-03-04T20:44:04.407000**
>
> Thanks!
>
> Here's a training graph from one of our 4000-epoch runs, but runs with a smaller number of epochs all had a similar shape. That up-surge at the end of the graph made it impossible to resist increasing the number of epochs even further! @sugupoko posted a very nice discussion about why more epochs improve the score.